

The Design and Analysis of the Symmetric Authenticated Tunneling Protocol

John G. Underhill
Quantum Resistant Cryptographic Solutions Corporation
`contact@qrcscorp.ca`

March 1, 2025

Abstract

The Symmetric Authenticated Tunneling Protocol (SATP) is a symmetric-only authenticated channel protocol for deployments in which endpoints can be provisioned with device-specific symmetric material before use and in which the operational cost, code footprint, or dependency structure of public-key key establishment is undesirable. SATP uses a deterministic cSHAKE-based hierarchy to derive server and device keys from a master secret, a two-message tunnel-establishment exchange to derive bidirectional channel state, and a post-tunnel client-authentication exchange mediated by a server callback. The encrypted transport is a stateful, per-direction authenticated stream cipher. In the analyzed reference instantiation it is RCS-256 keyed authenticated encryption using KMAC, with the serialized SATP packet header bound as associated data and strict in-order sequence checking applied before plaintext is released. This paper gives an implementation-aligned description of the SATP construction, states the assumptions under which its security claims hold, and analyzes server authentication, callback-mediated client authentication, channel confidentiality, ciphertext integrity, replay rejection, compromise boundaries, and operational limitations.

SATP does not provide public-key-style perfect forward secrecy. A party that records a handshake and later obtains the corresponding device leaf can recompute that session’s channel state, because the client nonce is transmitted in clear and the channel state is a deterministic function of the leaf, the configuration string, and that nonce. SATP instead provides single-use leaf separation, deterministic compromise containment, and efficient post-quantum-oriented symmetric security under the secrecy of provisioned material and the security of SHAKE/cSHAKE, KMAC, SCB, and the authenticated stream cipher. Server authentication provides implicit binding of the cleartext handshake fields, because modification of those fields causes the two endpoints to derive mismatched channel state and causes validation to fail.

Contents

1	Introduction	4
1.1	Context and Motivation	4
1.2	Contributions	4
1.3	Organization	4
2	Protocol Architecture	5
2.1	Roles and Trust Assumptions	5
2.2	Identity Layout	5
2.3	Master Key	5
2.4	Server Key	6
2.5	Device Key Record	6
2.6	Credential Verifier	6
3	Message Format and State Machines	7
3.1	Packet Header	7

3.2	Message Types	7
3.3	Channel-State Derivation	7
3.4	Client State Machine	8
3.5	Server State Machine	8
3.6	Pseudocode	8
4	Cryptographic Model and Assumptions	9
4.1	Primitive Assumptions	9
4.2	Adversary Capabilities	10
4.3	Freshness and Key-Use Assumptions	10
4.4	State and Time Assumptions	10
5	Security Definitions	10
6	Security Analysis	11
6.1	Server Authentication	11
6.2	Client Authentication	12
6.3	Channel Confidentiality	12
6.4	Ciphertext Integrity	12
6.5	Replay and Ordering	13
6.6	Compromise Boundaries	13
7	Replay, Freshness, Failure, and Termination Behavior	13
7.1	Established-Session Replay	13
7.2	Handshake Replay	13
7.3	Failure Paths	14
7.4	Termination Signaling	14
7.5	Denial-of-Service Considerations	14
8	Cryptanalysis and Robustness	14
8.1	Domain Separation	14
8.2	Metadata Exposure	14
8.3	Credential Exposure	15
8.4	Branch Compromise	15
8.5	Randomness Failure	15
9	Parameters and Implementation Considerations	15
9.1	Analyzed Parameters	15
9.2	Implementation Alignment	16
9.3	SCB Cost Parameters	16
9.4	Serialization and Storage	16
9.5	Implementation Assurance	16
10	Related Work and Positioning	16
10.1	Kerberos	16
10.2	TLS 1.3	17
10.3	AEAD Interfaces	17
10.4	Mobile AKA-Style Systems	17
11	Deployment Guidance	17
11.1	Provisioning	17
11.2	Persistent Leaf Consumption	17
11.3	Credential Handling	17
11.4	Network Exposure	17

12 Limitations and Future Work	18
12.1 No Perfect Forward Secrecy	18
12.2 No Explicit Transcript Hash	18
12.3 No Server Nonce	18
12.4 Positional Direction Separation	18
12.5 Strict Ordering	18
12.6 Callback Trust Boundary	18
13 Conclusion	18

1 Introduction

1.1 Context and Motivation

Symmetric trust infrastructures remain important in environments where devices are enrolled under a common administrative authority and receive secret material before deployment. Examples include controlled infrastructure networks, embedded access devices, industrial systems, field equipment, and systems where an operator can provision devices through a trusted channel. In such environments, symmetric cryptography can provide strong post-quantum security margins with low runtime cost, small messages, and compact implementation profiles. These properties are significant where public-key operations, certificate validation, or post-quantum key encapsulation mechanisms are operationally expensive.

SATP is designed for this model. It establishes an encrypted and authenticated duplex channel without asymmetric cryptography, public certificates, or online key distribution servers. A master key derives server or branch keys, server keys derive device key trees, and a client consumes one device leaf for each tunnel-establishment attempt. The server reconstructs the active device leaf from the server derivation key and the device identity string supplied by the client. A client-generated nonce is then mixed with the selected device leaf and the protocol configuration string to derive directional channel state. The server proves possession of the required branch material by returning an encrypted validation value. Client authentication occurs after the encrypted tunnel is raised and is enforced by the server application callback supplied to the SATP server routine.

The resulting design is not an authenticated key exchange in the same sense as TLS 1.3 or a KEM-based protocol. It does not negotiate algorithms, it does not transmit a server nonce, and it does not provide perfect forward secrecy against later compromise of the device leaf or branch key. It is a deterministic symmetric channel protocol whose security is bounded by the secrecy and lifecycle management of provisioned material. The purpose of this paper is to define that model precisely and to analyze the security properties that the construction provides, neither more nor less.

1.2 Contributions

This paper makes the following contributions.

- It gives an implementation-aligned formal specification of SATP, including the master-to-server derivation, server-to-device derivation, two-message tunnel establishment, post-tunnel authentication exchange, packet-header format, identity layout, and strict sequence-processing rules.
- It defines a security model for a symmetric provisioned tunnel protocol, distinguishing server authentication, callback-mediated client authentication, channel confidentiality, ciphertext integrity, replay rejection, and compromise containment.
- It presents security arguments with explicit advantage terms that reduce the main channel properties to framed cSHAKE pseudorandomness and to the confidentiality and integrity of the authenticated stream cipher, under the assumptions that device leaves are not reused and that callback authentication is correctly enforced by the server application.
- It identifies the protocol's security boundaries. SATP provides single-use leaf separation and deterministic session-key separation; it does not provide perfect forward secrecy against later disclosure of the selected leaf, the server derivation key, or the branch secret.
- It compares SATP with Kerberos, TLS 1.3 pre-shared-key deployments, and mobile AKA-style symmetric trust models, emphasizing differences in online infrastructure, transcript binding, forward secrecy, and provisioning assumptions.

1.3 Organization

Section 2 describes the SATP architecture, key hierarchy, and identity layout. Section 3 specifies the message formats and state machines. Section 4 defines the cryptographic model and adversary capabilities. Section 5 states the security definitions. Section 6 gives the principal security arguments. Section 7 analyzes replay,

ordering, failure, and termination behavior. Section 8 discusses compromise boundaries and operational robustness. Section 9 addresses parameters and implementation considerations. Section 10 compares SATP with related protocols. Section 11 gives deployment guidance. Section 12 states limitations and future work. Section 13 concludes.

2 Protocol Architecture

2.1 Roles and Trust Assumptions

SATP has two online protocol roles: a client and a server. A provisioning authority exists offline and creates master, server, and device key records. The root or master authority is not part of the online tunnel-establishment exchange. The protocol assumes that provisioning is performed over a trusted or otherwise protected channel, that clients receive device records containing their key tree and the server validation secret, and that the server possesses the server derivation key needed to reconstruct device leaves.

The implemented server application also stores a device credential verifier. During the client-authentication phase, the first encrypted message sent by the client contains a device identifier and a 32-byte credential token. The server application callback uses the device identifier to fetch the stored verifier and calls the SATP passphrase-verification interface. The authentication mechanism is therefore part of the supplied server application and the SATP callback interface; it is not hard-wired inside the key-exchange function. This boundary places the credential store and callback inside the trusted computing base for client authentication.

The protocol assumes the following long-term secrets remain protected before any compromise events explicitly modeled in the analysis:

- the master derivation key **mdk**, held by the provisioning authority;
- the server derivation key **sdk**, held by the server;
- the server validation secret $\mathbf{ST}_c$, held by the server and provisioned to clients in that branch;
- each unused device leaf $K_{c,i}$, held by the client key-tree record and reconstructable by the server;
- each stored credential verifier used by the server authentication callback.

2.2 Identity Layout

The reference implementation uses a fixed binary identity layout. A device identity **did** is the concatenation

$$\mathbf{did} = \mathbf{domain}[2] \parallel \mathbf{branch}[2] \parallel \mathbf{epoch}[2] \parallel \mathbf{service}[2] \parallel \mathbf{device}[4],$$

totalling 12 bytes. A key identity **kid** extends **did** with a 4-byte key index,

$$\mathbf{kid} = \mathbf{did} \parallel \mathbf{keyindex}[4],$$

totalling 16 bytes. The key index selects a leaf of the device key tree and is the field incremented in big-endian order during key-tree generation and persistent key-state update. A server identity $\mathbf{sid} = \mathbf{domain}[2] \parallel \mathbf{branch}[2]$ is the 4-byte prefix of **did**, and the master identity $\mathbf{mid} = \mathbf{domain}[2]$ is the leading 2 bytes. The relation $\mathbf{sid} = \mathbf{kid}[0:4]$ is used by the client to check the server identifier returned in the authentication response.

2.3 Master Key

The master key record is denoted

$$M = (\mathbf{mdk}, \mid, \mathbf{exp}).$$

The field **mdk** is a 256-bit master derivation key sampled by the system random provider; it is not derived from any other value. The master identifier **mid** identifies the administrative root, and **exp** records the validity period. The master key is an offline provisioning object and is not used during tunnel establishment.

2.4 Server Key

A server key record is denoted

$$S = (\text{sdk}, \text{sid}, \text{ST}_c, \text{exp}).$$

The server derivation key is computed by cSHAKE-256 with the master derivation key as the message input, the SATP configuration string as the function-name input, and the server identifier as the customization input:

$$\text{sdk} := \text{cSHAKE256}_{X=\text{mdk}, N=\text{cfg}, S=\text{sid}}(256).$$

The server validation secret ST_c is sampled independently by the random provider. It is not derived from sdk . The server validation secret is copied into device records and is used to authenticate the server during the connection response.

2.5 Device Key Record

A device key record is denoted

$$D = (\text{KTree}, \text{kid}, \text{ST}_c, \text{exp}, \text{spass}).$$

The reference constant `SATP_KEY_TREE_COUNT` fixes the number of device leaves at 1024. Each leaf has length `SATP_DKEY_SIZE`, which is 32 bytes. For each index $j \in \{0, \dots, 1023\}$, the provisioning algorithm derives a device leaf by cSHAKE-256 with the server derivation key as the message input, an empty function-name input, and the kid value carrying index j as the customization input:

$$K_{c,j} := \text{cSHAKE256}_{X=\text{sdk}, N=\epsilon, S=\text{kid}_j}(256).$$

Because the full 16-byte kid_j is the customization input, each leaf is computed under a distinct framed input. The server reconstructs an active device leaf using the same derivation and the kid value transmitted in the Connect Request; it rejects an index that is not below `SATP_KEY_TREE_COUNT`. The server does not store client key trees and does not retain per-leaf consumption state in the base SATP runtime model.

2.6 Credential Verifier

Client authentication is callback-mediated. At provisioning, the server samples a 32-byte credential token c_{dev} and stores only its verifier

$$v_{\text{dev}} := \text{SCB}(c_{\text{dev}})$$

in the server credential store, indexed by the device identifier. Here SCB denotes the memory-hard credential-hashing function with the configured cost parameters. The token c_{dev} is delivered to the device operator out of band. The client application places c_{dev} in the `spass` field of the device key record before calling the SATP client connection function. The SATP client then encrypts the pair

$$(\text{did}, c_{\text{dev}})$$

inside the established tunnel and sends it as the first encrypted message after tunnel establishment. The server receive loop routes that first encrypted message to the application callback, which fetches v_{dev} for the presented did and accepts if and only if $\text{SCB}(c'_{\text{dev}}) = v_{\text{dev}}$, where c'_{dev} is the received token, using the constant-time comparison helper `satp_server_passphrase_hash_verify`.

Two consequences follow. First, the value transmitted by the client is the secret token itself, not a client-side hash of it. The server-side SCB verifier protects the credential store against offline recovery if the database is exfiltrated, but it does not protect the token against an adversary who already holds the channel keys. Second, offline-guessing resistance is governed jointly by the min-entropy of c_{dev} and the SCB cost parameters. In the reference application the token is a high-entropy random value, so entropy dominates; deployments that substitute lower-entropy human-chosen credentials must raise the SCB cost parameters accordingly.

3 Message Format and State Machines

3.1 Packet Header

Each SATP packet begins with a 21-byte header. The implemented wire order is

$$\text{Header} = \text{flag}[1] \parallel \text{msglen}[4] \parallel \text{sequence}[8] \parallel \text{utctime}[8].$$

The multi-byte fields are serialized in little-endian order. The message-length field gives the length of the payload following the header. For encrypted packets, the payload consists of the ciphertext followed by a 32-byte authentication tag produced by the keyed authenticated stream cipher. The serialized 21-byte header is bound to the cipher as associated data before encryption or decryption, so every header field, including the flag, length, sequence, and timestamp, is integrity-protected for encrypted packets.

3.2 Message Types

The implemented exchange uses the online messages listed in Table 1. All encrypted messages are carried under the single `encrypted_message` packet flag. Dedicated authentication-request and authentication-response flags are defined in the header enumeration, but the implemented authentication request and response are transported as ordinary encrypted messages under the SATP channel; the dedicated flags are not emitted on the wire. Keepalive identifiers, timeout constants, and state structures present in the source are reserved and non-normative for this analysis. No rekeying, session resumption, renegotiation, or identity-rotation message is implemented.

Message	Direction	Protection	Payload
Connect Request	Client to Server	Plaintext	$\text{kid} \parallel N_h$
Connect Response	Server to Client	Authenticated encryption	H_c
Authentication Request	Client to Server	Authenticated encryption	$\text{did} \parallel c_{\text{dev}}$
Authentication Response	Server to Client	Authenticated encryption	sid
Application Data	Both directions	Authenticated encryption	Application payload
Error / Teardown	Both directions	Context-dependent	Error code

Table 1: SATP online message types.

3.3 Channel-State Derivation

The client samples a 32-byte nonce N_h . The channel-state expansion is cSHAKE-256 with the active device leaf as message input, the SATP configuration string as function-name input, and the client nonce as customization input:

$$X := \text{cSHAKE256}_{X=K_{c,i}, N=\text{cfg}, S=N_h}(1024).$$

In the reference instantiation the configuration string is the 25-byte ASCII value `r01-satp-rcs256-keccak256`. The 128-byte output X is partitioned, in implementation byte order, into two directional key/nonce pairs:

$$X = K_{c \rightarrow s} \parallel N_{c \rightarrow s} \parallel K_{s \rightarrow c} \parallel N_{s \rightarrow c},$$

where each component is 32 bytes. The first pair keys the client-to-server direction; the second pair keys the server-to-client direction. The initiating client installs bytes 0–63 as its transmit state and bytes 64–127 as its receive state. The server installs the same two segments in the opposite roles. Direction separation is positional rather than label-based: no explicit role or direction string is mixed into X . A standards-oriented variant could add explicit direction labels; the reference model follows the implemented ordering above.

The validation value is cSHAKE-256 with the client nonce as message input, the selected device leaf as function-name input, and the server validation secret as customization input:

$$H_c := \text{cSHAKE256}_{X=N_h, N=K_{c,i}, S=\text{ST}_c}(256).$$

This is a cSHAKE-framed computation over distinct input positions. It is not modeled as ordinary concatenation SHAKE over $N_h \parallel K_{c,i} \parallel \text{ST}_c$. Because the nonce is public and the secrets are in the cSHAKE function-name and customization fields, the security argument relies on pseudorandomness of the full SP 800-185 framed cSHAKE input when high-entropy secrets are absorbed in any framed input position, not only in the message input.

3.4 Client State Machine

The client state machine is as follows.

1. The client loads a device record and selects the leaf indexed by the current key index. A leaf whose bytes have already been erased is rejected. The transmit and receive sequence counters are initialized to zero.
2. The client samples N_h , computes X , installs the directional cipher states, computes H_c , and transmits a plaintext Connect Request containing $\text{kid} \parallel N_h$.
3. The client receives a Connect Response. It validates the header, binds the serialized response header as associated data, decrypts and authenticates the response, and compares the recovered value to the locally computed H_c .
4. If the validation value matches, the server is authenticated under the assumption that only the legitimate server can reconstruct $K_{c,i}$ and knows ST_c . The encrypted tunnel is live.
5. The client sends an encrypted Authentication Request containing $\text{did} \parallel c_{\text{dev}}$. It accepts the Authentication Response only if the decrypted server identifier equals the sid prefix $\text{kid}[0:4]$ embedded in the device identity.
6. After successful authentication, subsequent encrypted messages are application data and must pass the sequence, timestamp, and authentication checks.

3.5 Server State Machine

The server state machine is as follows.

1. The server accepts a socket connection and reads a Connect Request.
2. The server validates the header, extracts kid and N_h , reconstructs $K_{c,i}$ from sdk and kid , computes X , installs the directional cipher states in the inverse roles, computes H_c , and sends H_c encrypted as the Connect Response.
3. The server receive loop treats the first encrypted message after tunnel establishment as the Authentication Request. The decrypted plaintext is passed to the configured authentication callback.
4. If the callback returns success, the server sends an encrypted Authentication Response containing sid , marks the connection authenticated, and accepts later encrypted packets as application data.
5. If authentication fails, or a packet fails the sequence, timestamp, or authentication check, the server terminates the connection state.

The authenticated state is therefore a runtime property enforced by the server receive loop and the callback, not an additional cryptographic transcript message inside the key-exchange function. Until the callback accepts, the server delivers no decrypted application message to the application receive callback.

3.6 Pseudocode

Algorithm 1: SATP client tunnel establishment.

1. Select active leaf $K_{c,i}$ from KTree at index $\text{kid}[12:16]$; reject if erased.
2. Sample $N_h \leftarrow \{0, 1\}^{256}$.
3. Compute $X \leftarrow \text{cSHAKE256}_{X=K_{c,i}, N=\text{cfg}, S=N_h}(1024)$.

4. Split $X = K_{c \rightarrow s} \parallel N_{c \rightarrow s} \parallel K_{s \rightarrow c} \parallel N_{s \rightarrow c}$; install $(K_{c \rightarrow s}, N_{c \rightarrow s})$ as transmit and $(K_{s \rightarrow c}, N_{s \rightarrow c})$ as receive.
5. Compute $H_c \leftarrow \text{cSHAKE256}_{X=N_h, N=K_{c,i}, S=\text{ST}_c}(256)$.
6. Send Connect Request (kid, N_h) as plaintext.
7. Receive Connect Response; bind serialized response header as associated data; decrypt and verify.
8. Abort if authentication fails or the decrypted value differs from H_c .
9. Send encrypted Authentication Request $(\text{did}, c_{\text{dev}})$.
10. Receive and decrypt Authentication Response sid' ; accept only if $\text{sid}' = \text{kid}[0:4]$.

Algorithm 2: SATP server tunnel establishment.

1. Receive Connect Request (kid, N_h) ; validate header.
2. Recover $K_{c,i} \leftarrow \text{cSHAKE256}_{X=\text{sdk}, N=\epsilon, S=\text{kid}}(256)$; reject if index ≥ 1024 .
3. Compute $X \leftarrow \text{cSHAKE256}_{X=K_{c,i}, N=\text{cfg}, S=N_h}(1024)$.
4. Split X ; install $(K_{c \rightarrow s}, N_{c \rightarrow s})$ as receive and $(K_{s \rightarrow c}, N_{s \rightarrow c})$ as transmit.
5. Compute $H_c \leftarrow \text{cSHAKE256}_{X=N_h, N=K_{c,i}, S=\text{ST}_c}(256)$.
6. Send authenticated-encrypted Connect Response containing H_c .
7. Receive first encrypted message; decrypt and verify; pass plaintext to authentication callback.
8. Abort if the callback rejects.
9. Otherwise send encrypted Authentication Response containing sid and mark the connection authenticated.

4 Cryptographic Model and Assumptions

4.1 Primitive Assumptions

Framed cSHAKE as a PRF. The analysis models cSHAKE-256 and SHAKE-256 according to the SHA-3 sponge framework standardized in FIPS 202 and SP 800-185. SATP uses cSHAKE as a framed keyed sponge: security requires that when one or more high-entropy secret values are absorbed into the cSHAKE input frame, whether in the message input X , the function-name input N , or the customization input S , the resulting mapping on the remaining adversary-visible inputs is computationally indistinguishable from a pseudorandom function, provided the SP 800-185 framing is implemented exactly and the same framed tuple is not reused for an unrelated purpose. For an adversary making q queries and running in time t we write this advantage as $\text{Adv}_{\text{prf}}(t, q)$.

This assumption is stated at the level required by the implemented validation value. In the channel-state expansion the secret leaf appears in X ; in the validation value the public nonce appears in X while the secret leaf and validation secret appear in N and S . The proof therefore relies on pseudorandomness of the complete cSHAKE-framed input, not on an informal rule that secrets must appear only in the message input.

Authenticated stream cipher. The transport primitive is a stateful, per-direction authenticated stream cipher. In the analyzed reference instantiation it is RCS-256 with KMAC authentication: the analyzed configuration uses a 256-bit key, a 256-bit nonce, and a 256-bit tag, all installed once per direction from X . The cipher state for each direction is persistent across packets: the keystream and running authentication state advance with every transformed packet, and the serialized header is bound as associated data on each call. We model this primitive as a stateful AEAD that provides confidentiality under chosen-plaintext attack and integrity of ciphertexts against adaptive forgeries at the receiver's current state position. We write the corresponding advantages as $\text{Adv}_{\text{ind-cpa}}$ and $\text{Adv}_{\text{int-ctxt}}$.

A compile-time alternative instantiates the transport as AES-256-GCM through cipher-abstraction macros. That alternative is not the configuration analyzed in this paper. References below to the AEAD or the cipher denote the RCS-256/KMAC instantiation unless stated otherwise.

Credential-hashing function. SCB is modeled as a memory-hard credential-hashing function parameterized by CPU- and memory-cost constants. Its role in this paper is confined to the client-authentication callback: the server stores $v_{\text{dev}} = \text{SCB}(c_{\text{dev}})$ and verifies a received token c'_{dev} by checking $\text{SCB}(c'_{\text{dev}}) = v_{\text{dev}}$ in constant time. We do not assert that any specific default cost parameter yields a universal offline-guessing cost; the relevant guessing advantage $\text{Adv}_{\text{guess}}$ is a function of token min-entropy and configured cost.

4.2 Adversary Capabilities

The adversary controls the network. It may read, replay, delay, drop, inject, and modify packets. It may open sessions with the server and cause the client or server to process malformed messages. It may obtain complete transcripts of finished sessions. It may later compromise selected long-term values through explicit compromise oracles. The adversary is computationally bounded but may use quantum search against symmetric keys; accordingly, 256-bit symmetric keys are interpreted as providing approximately 128-bit brute-force resistance under Grover-style search, and all advantage terms are understood in that post-quantum-adjusted sense.

Endpoint compromise, extraction of stored device records, branch-key compromise, and credential-database compromise are modeled separately from the ordinary network adversary and are quantified in Sections 6.6 and 8.

4.3 Freshness and Key-Use Assumptions

A SATP session is fresh for the main channel-security definitions if all of the following hold.

1. The active device leaf $K_{c,i}$ has not been used in another accepted session.
2. The client nonce N_h is sampled uniformly and is not repeated with the same $K_{c,i}$.
3. The device leaf remains secret during the session.
4. The server derivation key sdk and server validation secret ST_c remain secret during the session.
5. The authentication callback correctly rejects invalid credential pairs before application data is accepted.

If any assumption fails, the corresponding guarantee is narrowed. Reuse of $K_{c,i}$ with the same N_h recreates the identical channel state and hence reuses a keystream, which is catastrophic for confidentiality. Later compromise of $K_{c,i}$ permits recomputation of past channel states from recorded transcripts because N_h is public.

4.4 State and Time Assumptions

Established packets are accepted only when the received sequence number equals the next expected value. The implementation uses strict in-order sequencing, not a sliding replay window. The 64-bit timestamp field is validated against a bounded window of ± 60 seconds in the reference configuration. The timestamp mechanism provides coarse freshness filtering, but it depends on local clock correctness and is not a substitute for unique keys, unique nonces, and strict sequence checking.

5 Security Definitions

Definition 1 (Server authentication). *A client session satisfies server authentication if, whenever the client accepts a Connect Response for cleartext fields (kid, N_h) , the accepted response is consistent with generation under channel state derived from the selected device leaf and with the validation secret ST_c associated with the provisioned branch, except with negligible probability. The definition is implicit: it authenticates possession of the required branch material and binds the cleartext fields through derived state, but it does not by itself assert public-key-style identity semantics or a transcript-bound AKE.*

Definition 2 (Callback-mediated client authentication). *A server session satisfies client authentication if application data is delivered to the receive callback only after the first encrypted client message has been accepted by the configured authentication callback, and that callback accepts only a token c'_{dev} for the presented did such that $\text{SCB}(c'_{\text{dev}}) = v_{\text{dev}}$ for the stored verifier v_{dev} .*

This definition explicitly includes the application callback and credential store in the trusted computing base. A deployment that supplies a permissive callback, treats a missing record as a zero-valued credential, or stores low-entropy credential material does not satisfy the definition.

Definition 3 (Channel confidentiality). *Let $b \in \{0, 1\}$ be a hidden bit. The adversary may choose equal-length plaintexts M_0, M_1 for a fresh, authenticated SATP session. The challenger encrypts M_b under the session's transmit state and returns the ciphertext. SATP provides channel confidentiality if every efficient adversary guesses b with advantage bounded by the advantage of distinguishing the channel-state expansion from random plus the advantage against the AEAD confidentiality experiment.*

The definition applies after the session state is installed and after the authentication model for the role under consideration has been satisfied. It does not hide the Connect Request identity or client nonce, both of which are transmitted in clear.

Definition 4 (Ciphertext integrity). *A ciphertext-integrity adversary wins if it causes a fresh SATP session to accept, at the receiver's current state position, an encrypted packet that was not produced by the legitimate peer for that position. SATP provides ciphertext integrity if this probability is bounded by the AEAD ciphertext-integrity advantage plus negligible terms arising from header ambiguity.*

Definition 5 (Strict replay rejection). *A SATP receiver rejects a replayed established-session packet if the replayed packet has a sequence number different from the receiver's next expected value, or if its authenticated header or ciphertext fails verification, or if its timestamp falls outside the validity window. SATP provides strict in-order replay rejection when every accepted established packet increments the receiver sequence state exactly once and no stale packet can be accepted after that increment.*

Definition 6 (Single-use leaf containment). *A SATP deployment provides single-use leaf containment if compromise of an unused device leaf exposes at most sessions established with that leaf, and if unrelated leaves remain computationally hidden under the cSHAKE derivation from sdk so long as sdk remains secret.*

This containment property is weaker than perfect forward secrecy; it is the relevant compromise notion for a deterministic symmetric key-tree protocol.

6 Security Analysis

Throughout this section, q_s denotes the number of sessions the adversary causes, q_p the number of established packets per session, and t its running time.

6.1 Server Authentication

Theorem 1 (Server authentication and handshake binding). *Assume framed cSHAKE pseudorandomness under the SATP input positions, assume the authenticated stream cipher releases plaintext only after successful tag verification, assume $K_{c,i}$ is known only to the legitimate client and reconstructable only by the legitimate server, and assume ST_c is known only to legitimate participants in the branch. Then the probability that a client accepts a Connect Response for cleartext fields (kid, N_h) without an entity possessing the corresponding branch material having generated a response consistent with those fields is bounded by*

$$\text{Adv}_{\text{prf}}(t, q_s) + q_s \cdot \text{Adv}_{\text{int-ctxt}}(t) + q_s \cdot 2^{-256}.$$

Proof. Fix a client session with cleartext fields (kid, N_h) . The client computes

$$X = \text{cSHAKE256}_{X=K_{c,i}, N=\text{cfg}, S=N_h}(1024), \quad H_c = \text{cSHAKE256}_{X=N_h, N=K_{c,i}, S=\text{ST}_c}(256).$$

To accept, the client must decrypt and verify the Connect Response under the receive state derived from X and recover a plaintext equal to H_c .

First, replace the cSHAKE expansion keyed by the secret leaf with a random function. Across q_s sessions this changes the adversary's view by at most $\text{Adv}_{\text{prf}}(t, q_s)$. The directional keys and nonces installed by the client are then uniform and independent of the public fields.

Second, with uniform secret receive-state key material, causing the client to accept any chosen plaintext for the Connect Response requires producing a header/ciphertext pair that verifies at the receiver's initial position. By ciphertext integrity, the per-session probability of such a forgery is at most $\text{Adv}_{\text{int-ctxt}}(t)$, giving $q_s \cdot \text{Adv}_{\text{int-ctxt}}(t)$ over all sessions.

Third, acceptance also requires that the recovered plaintext equal H_c . The value H_c is a cSHAKE-framed function of the public nonce and the secret framed inputs $K_{c,i}$ and ST_c . Under the framed PRF assumption, an adversary lacking those secrets sees H_c as computationally indistinguishable from a uniform 256-bit value. The residual per-session prediction probability is therefore at most 2^{-256} , contributing $q_s \cdot 2^{-256}$ after a union bound.

Handshake binding follows because the keys installed by the client are a function of $(K_{c,i}, \text{cfg}, N_h)$ and the keys installed by the server are a function of the leaf reconstructed from the received kid and the received nonce. If a network adversary alters either cleartext field, the endpoints derive different expansions except with negligible probability, so the encrypted response fails authentication or does not match the client's expected validation value. Acceptance therefore implies agreement on (kid, N_h) in the implemented two-message exchange. $\square$

6.2 Client Authentication

Theorem 2 (Callback-mediated client authentication). *Assume the server application supplies an authentication callback that accepts a presented pair $(\text{did}, c'_{\text{dev}})$ only if $\text{SCB}(c'_{\text{dev}})$ equals the stored verifier v_{dev} for did , that the verifier store is protected from unauthorized modification, and that the first encrypted client message is routed to that callback before application data is delivered. Then the probability that an adversary lacking a valid token causes the server to accept it as an authenticated client is at most*

$$q_s \cdot \text{Adv}_{\text{int-ctxt}}(t) + \text{Adv}_{\text{guess}}(t).$$

Proof. The server decrypts and verifies the first encrypted post-handshake packet under the established receive state and passes the plaintext to the callback. To have a chosen plaintext reach the callback, the adversary must either possess the established channel state or forge a packet that verifies at the receiver's position; the latter costs at most $\text{Adv}_{\text{int-ctxt}}(t)$ per session. Given a submitted token c'_{dev} for did , the callback accepts only if $\text{SCB}(c'_{\text{dev}}) = v_{\text{dev}}$. Lacking the legitimate token, acceptance requires guessing a preimage consistent with v_{dev} , corrupting the verifier store, or exploiting the callback implementation. Store integrity and callback correctness are assumptions, leaving the guessing term. $\square$

Remark 1. *Because the client transmits c_{dev} itself inside the tunnel, an adversary who already holds the channel keys learns the token directly. SCB hardening protects only the stored verifier against database exfiltration, not the token against a party with the session keys. Client authentication is therefore conditioned on channel confidentiality and the honesty of the receiving server.*

6.3 Channel Confidentiality

Theorem 3 (Channel confidentiality). *For a fresh authenticated SATP session, channel ciphertexts are indistinguishable under chosen-plaintext attack with advantage at most*

$$\text{Adv}_{\text{prf}}(t, q_s) + \text{Adv}_{\text{ind-cpa}}(t, q_p).$$

Proof. The directional states are derived from $X = \text{cSHAKE256}_{X=K_{c,i}, N=\text{cfg}, S=N_h}(1024)$. Under freshness, $K_{c,i}$ is unused elsewhere and N_h is unique for the leaf. Replacing the framed cSHAKE output by random changes the view by at most $\text{Adv}_{\text{prf}}(t, q_s)$ and yields uniform independent directional keys and nonces. With such key material, confidentiality of the q_p application packets reduces directly to the authenticated stream cipher's chosen-plaintext experiment with the serialized header as associated data. $\square$

6.4 Ciphertext Integrity

Theorem 4 (Ciphertext integrity). *For a fresh SATP session, the probability that an adversary causes acceptance of a packet at the receiver's current position that was not produced by the legitimate peer for that position is at most $\text{Adv}_{\text{int-ctxt}}(t)$ per packet, hence at most $q_p \cdot \text{Adv}_{\text{int-ctxt}}(t)$ per session.*

Proof. Before releasing plaintext, the receiver checks that the packet sequence equals the next expected value and that the timestamp is within the window, then binds the serialized header as associated data and invokes the authenticated decryptor at the current forward-only state position. Any modification of header bytes changes associated data and invalidates the tag; any packet not at the expected sequence is rejected before decryption; and any packet at the expected sequence requires a valid tag under the current state. Acceptance of a packet not produced by the legitimate peer for that position is therefore an AEAD ciphertext-integrity forgery. $\square$

6.5 Replay and Ordering

Theorem 5 (Strict in-order replay rejection). *SATP rejects established-session replay under strict in-order delivery and does not provide reordering tolerance.*

Proof. The receiver stores the last accepted sequence number and requires a candidate to equal that value plus one. On acceptance, the value is incremented and the cipher state advances one position. A replay of any earlier packet has a sequence at most the stored value and is rejected before plaintext release. A packet from a later position is rejected because it does not equal the next expected value and cannot verify because the forward-only cipher state has not reached that position. The same logic rejects genuinely reordered packets; no sliding-window state exists in the analyzed construction. $\square$

6.6 Compromise Boundaries

Lemma 1 (Leaf isolation). *Disclosure of one device leaf does not reveal any other device leaf unless the server derivation key is also disclosed, assuming framed cSHAKE pseudorandomness.*

Proof. Each leaf is $\text{cSHAKE256}_{X=\text{sdk}, N=\epsilon, S=\text{kid}_j}(256)$ with a distinct customization string. Under the PRF assumption, an output for one customization is independent of the output for another without the secret message input sdk ; observing one leaf gives only negligible advantage toward any other. $\square$

Lemma 2 (No forward secrecy under leaf disclosure). *Disclosure of the device leaf used for a recorded session allows recomputation of that session's channel state.*

Proof. The Connect Request transmits N_h in clear and cfg is public, while the channel state is $X = \text{cSHAKE256}_{X=K_{c,i}, N=\text{cfg}, S=N_h}(1024)$. An adversary that later obtains $K_{c,i}$ and has recorded N_h recomputes X and hence both directional states. SATP therefore does not provide perfect forward secrecy with respect to later disclosure of $K_{c,i}$; the same holds for disclosure of sdk , from which $K_{c,i}$ is recomputable for any known kid . $\square$

7 Replay, Freshness, Failure, and Termination Behavior

7.1 Established-Session Replay

Established encrypted packets receive three layers of rejection before application plaintext is released. First, the sequence number must equal the next expected value. Second, the timestamp must fall inside the ± 60 second window. Third, the authentication tag must verify under the serialized header as associated data at the current cipher-state position. The strict sequence check is the principal replay control; the timestamp bounds stale traffic but does not independently prove uniqueness. Because the per-direction cipher state is forward-only, a dropped packet desynchronizes the stream and all subsequent packets fail authentication. The design is therefore appropriate for reliable, ordered transports and is not appropriate for reordering transports without an additional ordering layer.

7.2 Handshake Replay

The Connect Request contains $\text{kid} \parallel N_h$ in clear, and the server derives session state deterministically from $K_{c,i}$, cfg , and N_h . A replayed Connect Request can therefore cause the server to recreate the same channel state if the timestamp window accepts the request and the server keeps no consumed-index state. The base

construction maintains no server-side consumed-leaf record, so independent server-side rejection of replayed Connect Requests is not part of the analyzed behavior. A deployment requiring that property should maintain a consumed-index database or mix a server nonce into the derivation.

This replay condition is both an availability concern and a cryptographic state-recreation condition. A replayed request alone does not authenticate the adversary as a client and does not give application data to the server, but it can force duplicate server-side channel-state construction under the same leaf and nonce. Server-side consumed-index tracking or a server nonce removes this condition.

7.3 Failure Paths

Authentication failure, sequence failure, timestamp failure, malformed-header failure, and tag-verification failure all terminate the affected connection path. Sensitive temporary buffers used in key exchange and authentication must be securely erased where they are held in explicit temporary state. Long-term serialized provisioning records require external storage protection and secure lifecycle management; they are not erased by the protocol.

7.4 Termination Signaling

Two teardown paths exist. After the tunnel is established, a teardown or error message is sent as an authenticated encrypted packet carrying a one-byte error code, and the receiver accepts it only if its sequence is the next expected value, its timestamp is valid, and its tag verifies. Before the tunnel is established, a teardown is a plaintext one-byte error appended to a header and is not authenticated. An adversary can therefore inject a plaintext error to abort a not-yet-established connection; this is a connection-setup denial-of-service issue and cannot cause acceptance of forged application data.

7.5 Denial-of-Service Considerations

SATP is computationally inexpensive before authentication. This benefits constrained devices but creates an availability consideration for servers exposed to untrusted networks, because a server allocates connection state and receive buffers before client authentication completes and because pre-establishment teardown is unauthenticated. Implementations should apply socket timeouts, connection limits, rate limits, and conservative allocation policies, and should defer expensive credential verification until the encrypted tunnel is established and message structure has been validated.

8 Cryptanalysis and Robustness

8.1 Domain Separation

The construction uses cSHAKE framing in several positions. Server derivation uses `cfg` as function name and `sid` as customization over secret `mdk`. Device-leaf derivation uses an empty function name and `kid` as customization over secret `sdk`. Channel-state derivation uses `cfg` as function name and N_h as customization over secret $K_{c,i}$. The validation value uses public N_h as message input, secret $K_{c,i}$ as function name, and secret ST_c as customization. These positions are part of the implemented construction and are modeled as cSHAKE framing rather than ordinary concatenation. A standards-oriented variant could strengthen separation further with explicit labels such as `satp.server`, `satp.device`, `satp.c2s`, and `satp.s2c`; the reference model uses the separation described here.

8.2 Metadata Exposure

The Connect Request is not encrypted. A passive observer learns the device key identity and client nonce. This is a deliberate tradeoff of the two-message design. Later authentication and application payloads are encrypted and authenticated, but packet length, timing, and connection metadata remain visible at the transport layer.

8.3 Credential Exposure

The client transmits its credential token over the encrypted SATP channel. If the device record is compromised, the adversary may obtain unused device leaves, ST_c , and the credential token. This is endpoint compromise and is outside ordinary network security. Deployments should store device records in secure storage, rotate device credentials after suspected extraction, and treat loss of ST_c as a branch-level event.

8.4 Branch Compromise

Disclosure of `sdk` permits derivation of all device leaves for identities in the branch. If ST_c is also disclosed, an adversary can impersonate the branch server to devices that trust that secret. Because N_h is visible in recorded Connect Requests, disclosure of `sdk` permits recovery of past channel states for recorded sessions whose device identities are known. Epoch rotation and branch revocation are therefore necessary operational controls.

8.5 Randomness Failure

If a client reuses N_h with the same $K_{c,i}$, the channel state and keystream repeat, which breaks confidentiality. If the device leaf is consumed exactly once, nonce reuse with the same leaf cannot occur through ordinary operation. The implementation must abort on random-generation failure and must persist leaf consumption in a manner resilient to power loss. The reference client erases the active leaf and advances the persisted key index before initiating the network connection, so a consumed leaf cannot be reused after a restart or crash; deployments should preserve that ordering.

9 Parameters and Implementation Considerations

9.1 Analyzed Parameters

Table 2 defines the SATP parameter set analyzed in this paper.

Parameter	Value	Meaning
Device leaf size	32 bytes	cSHAKE-derived device key $K_{c,i}$
Server validation secret	32 bytes	ST_c , shared between branch server and devices
Client nonce	32 bytes	N_h , sampled per establishment attempt
Channel-state output	128 bytes	X , split into four 32-byte fields
Key-tree count	1024	Number of leaves per device record
Packet header size	21 bytes	Flag, length, sequence, and UTC time
Header field order	little-endian	Serialization of multi-byte header fields
AEAD tag size	32 bytes	RCS-256/KMAC-256 authentication tag
AEAD nonce size	32 bytes	Per-direction nonce field of X
AEAD key size	32 bytes	Per-direction key field of X
Sequence rule	strict rxseq + 1	In-order delivery only; no sliding window
Time window	± 60 seconds	Packet timestamp freshness bound
Credential token size	32 bytes	Token c_{dev} presented to the server callback
Configuration string	<code>r01-satp-rcs256-keccak256</code>	25-byte value bound into derivations

Table 2: SATP parameter set analyzed in this paper.

9.2 Implementation Alignment

Table 3 maps modeled objects to implementation-level operations. The mapping supports traceability between the proof obligations and the concrete construction.

Modeled object	Symbol	Implementation-level operation
Server-key derivation	sdk	cSHAKE-256 over $(\text{mdk}, \text{cfg}, \text{sid})$ at key-tree setup
Device-leaf derivation	$K_{c,i}$	cSHAKE-256 over $(\text{sdk}, \epsilon, \text{kid}_j)$ per leaf
Channel-state expansion	X	cSHAKE-256 over $(K_{c,i}, \text{cfg}, N_h)$ in key exchange
Directional split	$K_{c \rightarrow s}, N_{c \rightarrow s}, K_{s \rightarrow c}, N_{s \rightarrow c}$	Byte offsets 0–63 and 64–127 of X
Server validation value	H_c	cSHAKE-256 over $(N_h, K_{c,i}, \text{ST}_c)$ using SP 800-185 framing
Channel AEAD	Enc/Dec	RCS-256/KMAC-256 duplex states, one per direction
Header as AAD	–	21-byte serialized header bound on each packet
Replay control	–	strict $\text{rxseq} + 1$ check before tag verification
Freshness window	–	± 60 second timestamp check before tag verification
Credential verifier	v_{dev}	$v_{\text{dev}} = \text{SCB}(c_{\text{dev}})$, compared by the server callback
Single-use leaf	–	on-disk leaf erased and index advanced before connect

Table 3: Mapping between modeled objects and implementation-level operations.

9.3 SCB Cost Parameters

The implementation exposes SCB cost constants used by credential-derivation and verification helpers. In the reference build these are set to the lowest cost class. This choice is acceptable only when the credential token is a high-entropy random value; for such a token the offline-guessing term is already negligible without memory-hard stretching. Deployments that derive credentials from lower-entropy secrets such as human-chosen passphrases must raise these cost parameters accordingly. The security analysis of credential-derived authentication must use the actual deployment cost parameters and the entropy of the underlying secret.

9.4 Serialization and Storage

SATP serializes master, server, and device key records into fixed-size binary records. These records contain long-term secret material and must be protected by the deployment environment. The base serialization format is not an encrypted storage container and should not be treated as a complete secure-storage system. Device records should be held in tamper-resistant memory, a secure element, encrypted local storage, or an equivalent mechanism appropriate to deployment risk.

9.5 Implementation Assurance

Publication-grade validation should include known-answer and negative-path tests for the following cases: deterministic derivation vectors for sdk , $K_{c,i}$, X , and H_c ; replayed Connect Requests; replayed established packets; wrong credential pairs; stale and future timestamps; sequence gaps; malformed length fields; altered header associated data; and corrupted authentication tags. These artifacts are validation requirements for an independently reproducible analysis, not additional assumptions of the protocol proof.

10 Related Work and Positioning

10.1 Kerberos

Kerberos V5 is a symmetric-key network authentication protocol that relies on an online key distribution center issuing tickets and session keys. SATP uses no ticket-granting service during tunnel establishment;

it relies on prior provisioning and deterministic server-side reconstruction of device leaves. This reduces online infrastructure but transfers responsibility to provisioning, credential storage, and device key-state management.

10.2 TLS 1.3

TLS 1.3 provides an authenticated transport protocol with transcript binding, negotiation, and support for ephemeral Diffie–Hellman and pre-shared-key modes. In ephemeral configurations TLS 1.3 provides forward secrecy that SATP does not claim. SATP removes public-key operations, certificate processing, and negotiation from the online path; the cost is narrower security, since compromise of branch material or a recorded device leaf can affect recorded sessions.

10.3 AEAD Interfaces

RFC 5116 defines an interface for authenticated encryption with associated data. SATP follows this model at the packet layer by authenticating the serialized header as associated data. The design adheres to the principle that unencrypted framing fields that influence decryption or state transitions must still be integrity-protected. The nonce-respecting requirement of AEAD interfaces is handled in SATP by fixing a per-direction nonce within a single forward-only channel state and by requiring that a $(K_{c,i}, N_h)$ pair not be reused.

10.4 Mobile AKA-Style Systems

Mobile authentication systems also rely on pre-provisioned symmetric secrets and challenge-response procedures. SATP resembles these systems in its use of provisioned device material, but differs in that the result of authentication directly initializes an application tunnel rather than merely authorizing network access, and in that replay and ordering checks are placed in the tunnel packet layer.

11 Deployment Guidance

11.1 Provisioning

Provisioning must ensure that device identifiers are unique within the server branch, that key-tree records are generated with the correct server derivation key, and that the client receives the correct ST_c for its branch. The provisioning process should initialize device-key and server-side credential records atomically so that a device key and its verifier cannot diverge.

11.2 Persistent Leaf Consumption

Single-use leaf behavior depends on persistence. A client must advance serialized device-key state so that a consumed leaf cannot be reused after restart or power loss. If the persistent update fails, the connection attempt must be aborted. A server-side consumed-index database may be added by deployments that require independent rejection of replayed Connect Requests.

11.3 Credential Handling

The server authentication callback is security critical. It must validate message length, fetch the verifier corresponding to the supplied device identifier, compare using the constant-time SATP verification helper, and reject when lookup fails. A missing record must be treated as rejection, never as a zero-valued credential. Credential buffers should be erased after use.

11.4 Network Exposure

Internet-facing deployments should apply admission controls, timeouts, and connection limits. The protocol is most naturally suited to controlled-access networks, embedded systems, and private infrastructure where endpoint enrollment is administered centrally.

12 Limitations and Future Work

12.1 No Perfect Forward Secrecy

SATP does not provide perfect forward secrecy. A recorded Connect Request contains N_h , and the channel state is deterministic in $(K_{c,i}, \text{cfg}, N_h)$. Later disclosure of $K_{c,i}$ or sdk therefore allows reconstruction of past sessions. The defensible claim is single-use leaf containment and deterministic session separation, not public-key-style forward secrecy.

12.2 No Explicit Transcript Hash

SATP does not maintain a transcript hash analogous to TLS 1.3 or Noise. The cleartext kid and N_h are bound indirectly: they determine device-leaf reconstruction and channel-state derivation, and any modification prevents the encrypted confirmation value from authenticating under the client's expected receive state. This provides implicit handshake-field binding for the implemented two-message exchange, but it is weaker and less extensible than an explicit transcript hash. A standards-oriented revision should bind a canonical transcript hash into both the channel expansion and confirmation value.

12.3 No Server Nonce

The server contributes no fresh nonce to channel-state derivation. Session freshness rests on the client nonce and single-use leaf behavior. Adding a server nonce would remove the handshake-replay state-duplication concern and make the channel state depend on both endpoints.

12.4 Positional Direction Separation

The implemented key schedule separates channel directions by output position rather than by explicit labels. This is acceptable for the fixed implemented split but less robust than a labeled derivation in an extensible standard. A standards-oriented revision should add explicit direction labels for client-to-server and server-to-client key material.

12.5 Strict Ordering

SATP rejects out-of-order packets. This simplifies replay logic and suits reliable in-order streams, but it does not tolerate reordering. A datagram or lossy-transport variant would require a different replay-window and AEAD-state model.

12.6 Callback Trust Boundary

Client authentication is only as strong as the configured callback and credential store. A permissive callback, weak credential, or failed lookup not treated as rejection invalidates the client-authentication theorem. Future work may define a fully normative in-protocol credential record and verification procedure, removing the callback from the trusted computing base.

13 Conclusion

SATP is a compact symmetric tunnel protocol for controlled environments with pre-provisioned device secrets. It uses a cSHAKE-based deterministic key hierarchy, a two-message tunnel-establishment exchange, a forward-only authenticated-encryption channel with the serialized header as associated data, strict in-order sequence processing, and post-tunnel callback-mediated client authentication. Under the assumptions stated in this paper, SATP provides server authentication with implicit handshake-field binding, callback-mediated client authentication, channel confidentiality, ciphertext integrity, and strict in-order replay rejection, with advantage terms reducing the properties to framed cSHAKE pseudorandomness and to the confidentiality and integrity of the RCS-256 AEAD.

The guarantees are intentionally scoped. SATP does not provide perfect forward secrecy, transcript-bound authenticated key exchange, reordering-tolerant replay windows, algorithm negotiation, or anonymity. Its value is efficient symmetric security in systems where provisioning is already part of the deployment model and where the cost or complexity of public-key mechanisms is undesirable. Correct deployment requires strong protection of device records, reliable leaf consumption, credential-store integrity, callback correctness, and operational rate limiting. Within those constraints, SATP is a clear and analyzable symmetric alternative for post-quantum-oriented secure channels.

References

- [1] QRCS Corporation. *Symmetric Authenticated Tunneling Protocol (SATP) Technical Specification*. Quantum Resistant Cryptographic Solutions Corporation, 2025. https://www.qrcscorp.ca/documents/satp_specification.pdf.
- [2] QRCS Corporation. *SATP Source Code Repository*. Quantum Resistant Cryptographic Solutions Corporation. <https://github.com/QRCS-CORP/SATP>.
- [3] National Institute of Standards and Technology. *FIPS 202: SHA-3 Standard: Permutation-Based Hash and Extendable-Output Functions*. U.S. Department of Commerce, 2015. <https://doi.org/10.6028/NIST.FIPS.202>.
- [4] J. Kelsey, S. Chang, and R. Perlner. *NIST SP 800-185: SHA-3 Derived Functions: cSHAKE, KMAC, TupleHash, and ParallelHash*. National Institute of Standards and Technology, 2016. <https://doi.org/10.6028/NIST.SP.800-185>.
- [5] D. McGrew. *RFC 5116: An Interface and Algorithms for Authenticated Encryption*. Internet Engineering Task Force, 2008. <https://www.rfc-editor.org/rfc/rfc5116>.
- [6] M. Bellare and C. Namprempre. *Authenticated Encryption: Relations among Notions and Analysis of the Generic Composition Paradigm*. Cryptology ePrint Archive, Report 2000/025, 2000. <https://eprint.iacr.org/2000/025>.
- [7] E. Rescorla. *RFC 8446: The Transport Layer Security (TLS) Protocol Version 1.3*. Internet Engineering Task Force, 2018. <https://www.rfc-editor.org/rfc/rfc8446>.
- [8] C. Neuman, T. Yu, S. Hartman, and K. Raeburn. *RFC 4120: The Kerberos Network Authentication Service (V5)*. Internet Engineering Task Force, 2005. <https://www.rfc-editor.org/rfc/rfc4120>.
- [9] G. Bertoni, J. Daemen, M. Peeters, and G. Van Assche. *On the Security of the Keyed Sponge Construction*. Symmetric Key Encryption Workshop, 2011.
- [10] P. Rogaway. Authenticated-encryption with associated-data. In *Proceedings of the 9th ACM Conference on Computer and Communications Security*, pages 98–107, 2002. <https://doi.org/10.1145/586110.586125>.